PRAHARI

Proof-carrying Reasoning And Hardening for Autonomous Remediation of Intrusions

An air-gap-deployable cyber-reasoning system that finds, patches, and **proves** — with no human in the loop.

01 THE PROBLEM

WHY THE CURRENT PATCH CYCLE CANNOT HOLD

- Armed Forces web and API software — logistics portals, C2 services, internal REST APIs — is patched on a human cycle measured in weeks. Adversary exploitation windows are measured in hours.
- Commercial scanners find but never fix, bury analysts in false positives, and are cloud-only — a non-starter on a classified network.
- Every hour a known flaw stays live is an hour of exposed operational surface.
- The finale target is a simulated Armed Forces environment — the same shape of software, under the same constraint: no source code leaves the network.

02 THE GAP

WHY EXISTING CRS DESIGNS DO NOT TRANSFER

- Autonomous cyber-reasoning works on C/C++ because a segfault is a free ground-truth oracle. AddressSanitizer says "this is a real bug" — for free, with no ambiguity.
- Web and API services have no segfault. An SQL injection returns HTTP 200.
- So fuzzers are blind, findings are noisy guesses, and an LLM's "fix" cannot be verified. This is the gap PRAHARI is built to close.
- Bolting an LLM onto a scanner inherits the same blindness — it only rewrites what the scanner already guessed at.

03 THE SOLUTION

SUPPLY THE MISSING ORACLE, THEN THE MISSING PROOF

- PySan — a runtime taint sanitizer that gives Python services the oracle they never had: tainted input reaching a dangerous sink becomes a deterministic, replayable crash.
- Proof-carrying patches — no fix is accepted on model confidence. Each must clear four machine-checkable gates and ships with the evidence attached.
- Local-first reasoning. One laptop, fully offline, no cloud dependency.
- The result of a run is not an alert. It is a finding, a patch, and the evidence that the patch works — produced end to end without an operator.

A closed autonomous loop — the system does not stop at a finding, and does not stop at a patch.

CYBERKNIIGHTS Danish Dhanjal · Hardik Trikha

Detailed Methodology

01 INGEST & MAP

Clone the target. tree-sitter AST plus call graph. Routes lifted from OpenAPI / Swagger, Flask and FastAPI decorators, and Django URLconf.

OUT attack-surface inventory + call graph

02 STATIC PRE-FOCUS

Semgrep, CodeQL and custom taint rules rank every source-to-sink path. The LLM reads only the ranked slices, never the whole repository — the context economy that keeps PRAHARI lightweight.

OUT ranked candidate sink paths

03 AUTO-HARNESS SYNTHESIS

The model writes one fuzz driver per route, seeded from spec examples and recorded traffic. No human writes a harness — this is what lets PRAHARI be aimed at unfamiliar infrastructure.

OUT per-route fuzz drivers + seed corpus

◆ CORE INNOVATION

04 PySan TAINT ORACLE

Import-time hooks on dangerous sinks: cursor.execute, os.system, subprocess, pickle.loads, eval, open, ORM raw. Taint propagates from every request object. Reaching a sink is a deterministic crash.

OUT replayable PoV + full taint path

05 TRIAGE & DEDUP

PoVs are clustered by taint-path signature, mapped to CWE, and scored for severity by reachability and authentication requirement. Served entirely by the on-device 3B model.

OUT deduplicated, CWE-tagged, ranked findings

06 PATCH SYNTHESIS

N candidate minimal diffs, each constrained by the recorded taint path and ranked locally. Escalates to the Claude API only when every local candidate fails validation.

OUT candidate patch diffs

◆ THIS IS WHERE THE FIX IS PROVEN

07 PROOF HARNESS — FOUR GATES, ALL MUST PASS

A patch is accepted only when all four gates pass. Output: a signed proof bundle plus a patch PR.

- GATE 1 the PoV replays and is now blocked
- GATE 2 the existing test suite is 100% green
- GATE 3 10k benign requests: pre/post responses identical
- GATE 4 no new taint path introduced by the patch

FEEDBACK LOOP any gate fails → regenerate the patch with the failing gate as evidence — bounded at K iterations, then escalated for human review

IMPLEMENTATION STRATEGY

36-hour build order — the oracle is built first because nothing downstream works without it

H 00–06
PySan sink hooks and taint propagation, validated against a known-vulnerable Django app

H 06–16
Route mapping, harness synthesis, orchestrator state machine, local model tier

H 16–28
Patch synthesis loop with all four gates and the feedback path closed

H 28–36
Dashboard, proof-bundle signing, benchmark run against the seeded corpus

Technology Stack & System Architecture

TARGET LAYER

Flask · FastAPI · Django · Node / Express · Docker-containerised services · OpenAPI 3 / Swagger

▼

SENSING LAYER

Semgrep · CodeQL · tree-sitter · OpenAPI parser · Atheris · Hypothesis · schemathesis · coverage.py · PySan taint runtime

▼

REASONING LAYER

asyncio orchestrator (state machine) · TIER-1 LOCAL Ollama + Qwen2.5-Coder 3B/7B · TIER-2 CLOUD Claude API (escalation only) · FAISS RAG CWE + CVE + codebase · token / CPU budget meter

▼

ASSURANCE LAYER

pytest regression harness · differential replay engine · git apply / rollback · ed25519 proof-bundle signer · SARIF export

▼

OPS LAYER

FastAPI control plane · HTMX dashboard · SQLite findings store · Docker Compose · structured JSON audit log · runs fully offline

EQUIPMENT & FOOTPRINT

No specialised hardware. No cloud. No licence-bound tooling.

- Commodity x86 laptop
- 16 GB RAM
- 8 GB GPU — optional, for the local model only
- Everything in the stack is open source or already licensed

DESIGN TARGETS

≥ 90%
of decisions resolved on-device by the 3B local tier

< 6 GB
peak RAM for a full autonomous run

< 90 s
cold start to the first captured PoV

Deployable inside an air-gapped Armed Forces network, on hardware already in service.

Salient Features & Novelty

USP 1

THE MISSING ORACLE — PySan

Autonomous CRS works on C/C++ because AddressSanitizer turns a memory bug into a segfault — free ground truth. Web services have no such signal. PySan is the ASan-equivalent for Python: it propagates taint from every request into every dangerous sink, converting silent injection, path traversal, SSRF, unsafe deserialization and broken-authorization bugs into deterministic, replayable crashes.

```
tainted(request.args['q'])  reaches  cursor.execute()  ⇒  PoV, CWE-89
```

No existing tool pairs runtime taint tracking with autonomous patching.

USP 2

PROOF-CARRYING PATCHES

A patch is never accepted on model confidence. It must clear four machine-checkable gates, and it ships with the evidence attached:

`G1 PoV replay blocked` `G2 tests 100% green`
`G3 10k-request differential fuzz identical` `G4 no new taint path`

The brief asks us to prove the fix holds. This answers it literally — and the bundle is re-checkable by a change-control board without trusting us.

ZERO-HARNESS AUTONOMY

Point it at a repository and a base URL. It writes its own fuzz drivers from the spec — nothing to configure for unfamiliar infrastructure.

AIR-GAP FIRST, LIGHTWEIGHT

About 90% of decisions are resolved by a 3B on-device model plus deterministic analysers. Cloud is an optional escalation, never a dependency.

EXPLAINABLE & AUDITABLE

Every finding carries its taint path, PoV, CWE mapping and signed proof bundle — built to survive defence change-control review.

GENERALISES BY DESIGN

The oracle sits at framework level, not application level. New target, no new rules, no retraining, no re-tuning.

WHERE PRAHARI SITS	FINDS	PATCHES	PROVES	WEB / PYTHON	OFFLINE
Scanners — Snyk, Semgrep, Burp	✓	✗	✗	✓	partial
LLM autofix — Copilot Autofix	✓	✓	✗	✓	✗
AIxCC-class CRS — C/C++ only	✓	✓	✓	✗	✗
PRAHARI	✓	✓	✓	✓	✓

PRAHARI is the only point in this space that is autonomous, verifiable, web-native and air-gap-capable at the same time.

Final Deliverables

DELIVERABLES

1. **PRAHARI engine + CLI** — one-command Docker Compose run against any target repository and base URL
2. **PySan taint runtime** — released as a standalone, reusable Python library
3. **Proof-bundle spec + verifier** — anyone can re-check a patch without trusting the system that produced it
4. **Operator dashboard** — live findings, taint paths, patch diffs and gate results
5. **Benchmark report** — against a seeded-vulnerability corpus of Django/Flask apps plus real CVEs

PROOF-OF-CONCEPT — LIVE DEMONSTRATION

A vulnerability is injected into a running target; the operator touches nothing after this line:

```
$ prahari run --repo ./target --url http://127.0.0.1:8000
[map]     routes discovered ..... 34
[static]  ranked sink paths ..... 12
[pysan]   PoV captured   CWE-89   GET /api/units?q=
[patch]   candidate 2 of 4 accepted   (local 3B tier)
[gates]   G1 blocked  G2 118/118  G3 10k ok  G4 clean
[bundle]  proof a91f7c..  SIGNED  ->  patch PR opened
```

Illustrative output — every line above is produced by the pipeline on Slide 2.

PERFORMANCE OBJECTIVES

targets for the 36-hour finale — not results already measured

≥ 85%	≤ 5%	< 10 min	100%	< 6 GB	≥ 90%
detection rate on the seeded corpus	false-positive rate on confirmed findings	mean time from finding to proven patch	regression suite pass before a patch ships	peak RAM for a full autonomous run	of decisions resolved fully offline

FINALE READINESS

PRAHARI is aimed at a target by repository path and base URL alone — no per-application rules, no retraining, no reconfiguration. It can be pointed at the simulated Indian Armed Forces environment on day one, and every result it produces arrives with the evidence a reviewer needs to accept or reject it.

SUBMITTED AS

- GitHub repository source + Docker Compose
- Recorded 3-minute demonstration
- Benchmark & methodology report